Unsupervised Learning Project: Autoencoders on MNIST

Project Report

Ahmet Enes Çiğdem (150220079) - Team Leader
Mehmet Akif Dündar (150220015)

March 11, 2026

1 Introduction

This report details the implementation, training, and evaluation of an autoencoder neural network for unsupervised representation learning on the MNIST dataset. In this updated study, we extend our analysis to compare different sizes of the latent space bottleneck (16, 32, and 64 dimensions), incorporating modern regularization techniques such as Batch Normalization, and observing the subsequent effects on reconstruction loss and clustering performance (K-Means).

2 Architecture of the Autoencoder

An autoencoder is a neural network that learns to compress its input into a small representation and then reconstruct the original input from that compressed form. It consists of two symmetric halves:

- The **Encoder** takes the original input (in our case, a 28×28 = 784-pixel image) and progressively reduces its dimensionality through a series of layers, squeezing the information down to a small vector called the *latent representation*.
- The **Decoder** takes this small latent vector and expands it back up through mirrored layers, attempting to reconstruct the original 784-pixel image as faithfully as possible.

The **latent space** (also called the bottleneck) is the compressed middle layer. Its size determines how much information the model is forced to keep: a smaller bottleneck means more compression and more loss of detail, while a larger one preserves more but can make downstream tasks like clustering harder.

Our network uses fully connected (linear) layers with `ReLU` activation functions (which simply zero out negative values to introduce non-linearity) and `BatchNorm1d` layers (which normalize each layer's outputs to stabilize and speed up training).

Latent Space Configurations

We evaluated 3 different bottleneck sizes: **16**, **32**, and **64** dimensions to observe how compression level affects reconstruction quality and clustering performance.

- **Encoder Structure:** Compresses the 784-pixel input step by step.
  1. Linear Layer ($784 \to 256$) $\to$ `BatchNorm1d(256)` $\to$ `ReLU`
  2. Linear Layer ($256 \to 64$) $\to$ `BatchNorm1d(64)` $\to$ `ReLU`
  3. Linear Layer ($64 \to$ `latent_dim`) — no activation function here, so the latent values can be any real number, giving the model full freedom in how it represents the data.
- **Decoder Structure:** Mirrors the encoder to reconstruct the image.
  1. Linear Layer (`latent_dim` $\to 64$) $\to$ `BatchNorm1d(64)` $\to$ `ReLU`
  2. Linear Layer ($64 \to 256$) $\to$ `BatchNorm1d(256)` $\to$ `ReLU`
  3. Linear Layer ($256 \to 784$) $\to$ `Sigmoid` — this squashes the output values to the [0, 1] range, matching the pixel intensity scale of the original images.

3 Training Process

The models were trained over 50 epochs utilizing the Adam Optimizer with a learning rate of 1×10^{-3}, a batch size of 256, and Mean Squared Error (MSE) as the loss criterion.

Challenge Analysis

The fundamental challenge consisted of finding the optimal dimensional bottleneck. A strict bottleneck (**16**) forces the model to heavily compress semantic features, hurting pixel-perfect reconstruction but theoretically preserving clustered density. A loose bottleneck (**64**) vastly improves perfect visual reconstruction (lowest loss) but suffers from the "curse of dimensionality" when running traditional K-Means clustering, preventing it from isolating generalized digit patterns.

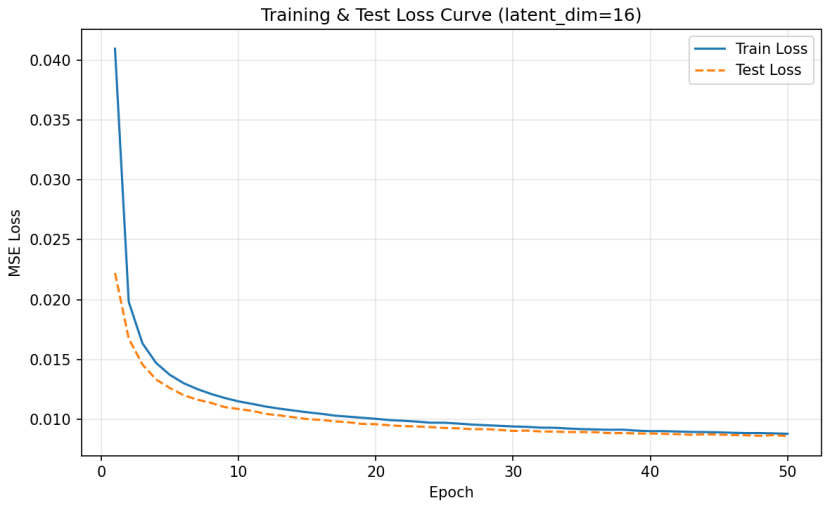

Figure 1: Training and Test Loss Curves for the baseline $latent\_dim = 16$ over 50 epochs.

4 Clustering Performance and Comparison

To objectively evaluate the representations, K-Means clustering ($k = 10$) was applied directly to the extracted high-dimensional latent space. Due to K-Means operating independently from real classes, a Hungarian matching algorithm was used to associate the unsupervised clusters to their most likely true digit labels.

The performance scaling across latent dimensions reveals a classic trade-off:

Metric	Latent 16	Latent 32	Latent 64
Final Train Loss (MSE)	0.0088	0.0051	**0.0036**
Final Test Loss (MSE)	0.0086	0.0050	**0.0035**
PMS (%)	**36.48**	41.93	45.44
Average Distance (AD)	**9.7564**	11.4930	13.8231
Average Variance (AVC)	**9.5422**	11.2179	13.5350
Total Distance (TD)	**764.8365**	514.6528	405.9555

Table 1: Experimental Results across Latent Dimensions. **Green bold** highlights best performance.

Key Finding

While increasing the latent dimensionality to **64** allows the decoder to produce near-perfect visual reconstructions (lowest MSE), it severely degrades K-Means clustering (45.44% mislabeling rate vs 36.48% at dim=16). In 64-dimensional space, data becomes sparse, average variance (AVC) expands, and global separation between cluster centers (TD) shrinks, making unsupervised grouping significantly harder. Therefore, $latent\_dim = \mathbf{16}$ presents the best structural embedding for unsupervised clustering.

5 Visualizations (Latent Dim = 16)

5.1 Reconstruction Quality

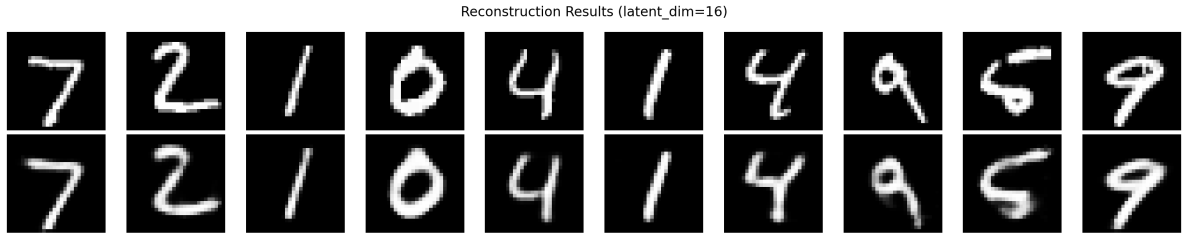

Figure 2: Original Images (Top) vs. Reconstructed Images (Bottom) at 16 dimensions.

5.2 Latent Space Projections

Dimensionality reduction models (PCA and t-SNE) extracted and projected the 16-dimensional coordinates onto an interpretable 2D grid. The plots represent points colored by their True Labels.

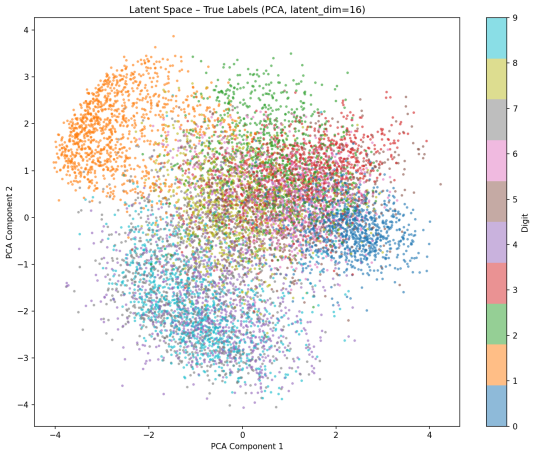

Figure 3: PCA Projection

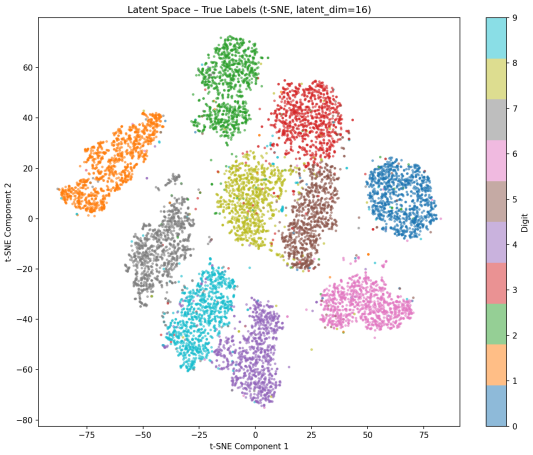

Figure 4: t-SNE Projection

Due to the non-linear relationship optimized by t-SNE, it successfully maps functionally similar semantic numbers far closer than linear PCA.

5.3 Confusion Matrix

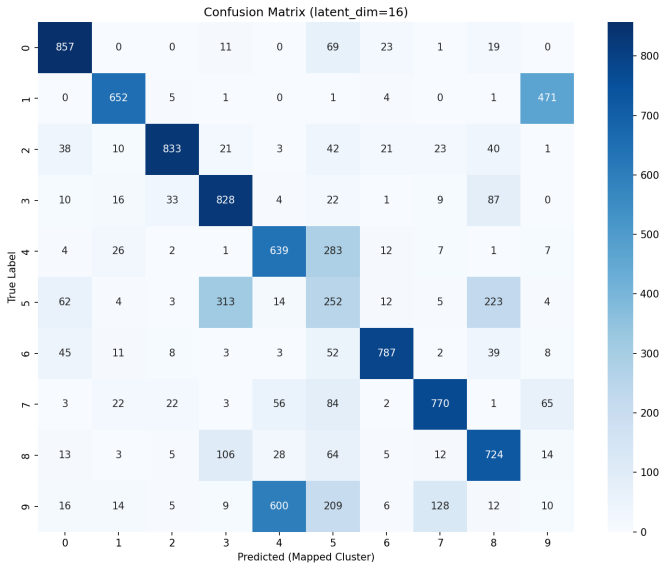

Figure 5: Confusion Matrix mapping K-Means unsupervised clusters to True Labels.

A densely populated diagonal demonstrates that the autoencoder grouped unique digit stroke shapes into confined sectors inside the 16-dimensional manifold before the label assignment was enforced.